

Natural Language Processing 2

Transformers

Fatemeh Dehrouyeh



FANSHAWE

Contents

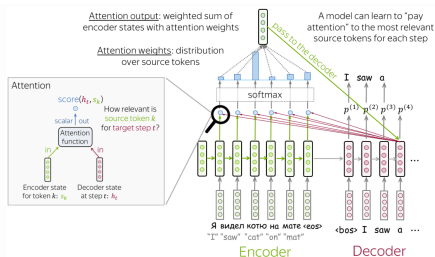
- 1 Attention
- 2 Transformer
- 3 Encoder-Decoder Overview

Current Section

- 1 Attention
- 2 Transformer
- 3 Encoder-Decoder Overview

Attention: A High-Level View

- Instead of squeezing the whole input into one vector, **attention** lets the decoder *focus on different parts of the input at each step*.
- At decoding step t :
 - Compare the decoder's state h_t with each encoder state s_k .
 - Compute **attention weights** (how relevant each word is to the decoder's current task (choosing the next output word.)).
 - Build a **context vector** = weighted sum of all encoder states.
- This context is passed to the decoder → helps pick the next word.



How Attention Works (Step by Step)

At each decoder step t , attention:

- ➊ **Takes input:** decoder state h_t and all encoder states $s_1, \dots, s_m$.
- ➋ **Computes scores:** for each source state s_k ,

$$\text{score}(h_t, s_k)$$

measures relevance of token k .

- ➌ **Normalizes scores:** apply softmax to get attention weights

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^m \exp(\text{score}(h_t, s_i))}$$

- ➍ **Builds context:** weighted sum of encoder states

$$c^{(t)} = \sum_{k=1}^m a_k^{(t)} s_k$$

- ➎ **Passes context $c^{(t)}$ to decoder** → used to predict the next word.

What Does an Attention Score Mean?

- At each decoding step t , we ask: *How relevant is source token s_k for predicting the next word?*
- One simple way: use the **dot product** between
 - decoder state h_t (what the model wants now), and
 - encoder state s_k (representation of source token k).
- If h_t and s_k point in a similar direction, their dot product is large $\rightarrow$ the model pays more attention to s_k .

Dot-product



$$h_t^T \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Intuition: Attention score is a **similarity measure** between what the decoder needs and what each source token provides.

What Makes Attention Powerful

Fixed-Vector Seq2Seq (RNNs)

- Compresses the whole source sentence into **one vector**.
- Decoder uses the **same vector** at every step.
- Long sentences can lose important information.

One fixed summary

Seq2Seq with Attention (Transformers)

- Keeps **all encoder hidden states**.
- Builds a **new context vector** at each step.
- Weights change based on the next word being generated.

Dynamic context

Key Difference

Attention lets the decoder focus on the most relevant source words at each decoding step.

Current Section

- 1 Attention
- 2 Transformer**
- 3 Encoder-Decoder Overview

What Is a Transformer?

A Transformer is a neural network architecture designed to process sequences, such as text.

Instead of reading words one by one, it looks at all words at the same time and uses attention to decide which words are most important for understanding the meaning.

This makes Transformers very effective for tasks like translation, text generation, summarization, and question answering.

Transformer: Attention is All You Need

- Introduced in 2017 (Vaswani et al., *Attention is All You Need*).
- **No recurrence, no convolutions** — built entirely on attention.
- Advantages:
 - Much faster to train (parallelizable).
 - Handles long sequences better.
 - Now the standard for NLP tasks (translation, language modeling, pretraining).
- New paradigm: instead of step-by-step recurrence, **all tokens can interact at once through attention**.
- **Parallelizable**: unlike RNNs, tokens can be processed at the same time.
- Forms the foundation of modern LLMs (e.g., BERT, GPT, PaLM, LLaMA).

Transformer Architecture

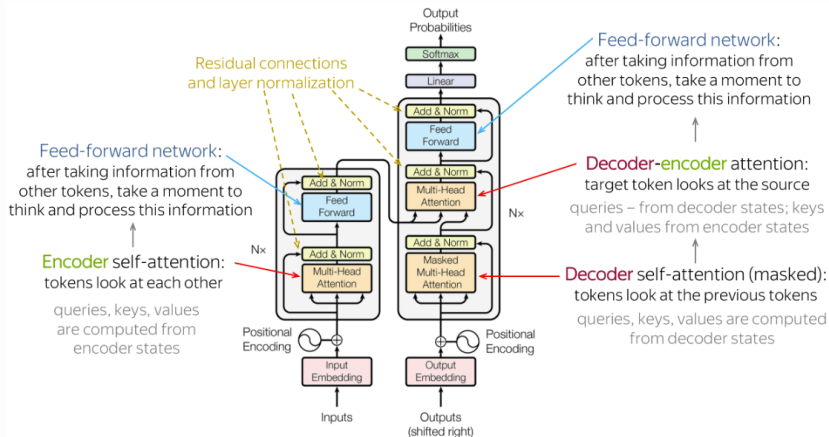


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Core Idea of the Transformer

Concept: Transformers process all words in a sequence at once and learn how each word relates to others through attention.

Main components:

- **Encoder:** Reads the input sequence and produces contextual representations.
- **Decoder:** Generates the output sequence word by word.
- Both use attention to focus on relevant parts of the sequence.

Example:

- For “The cat sat on the mat,” the model learns strong attention between “cat” and “sat,” and between “the” and “mat.”

Attention helps the model understand which words matter most to each other.

An Intuitive View of Attention

An easy way to think about Transformers is to imagine yourself reading a long sentence. Not every word is equally important. Some words carry more meaning, and your brain naturally pays more attention to them to understand the sentence.

Transformers do something similar. Through attention, they learn which parts of the sentence matter most at each moment. This helps them build a better understanding of the input and produce an output that makes sense.

Self-Attention: Tokens Look at Each Other

- So far, we saw **encoder–decoder attention (cross-attention)**:
 - From: one decoder state.
 - At: all encoder states.
- In **self-attention**, tokens interact within the *same sequence*.
 - From: each token's hidden state.
 - At: all other tokens' hidden states (in the same layer).
- Effect: every token gathers context from the entire sentence.
- This makes self-attention the core mechanism of the Transformer.

Query, Key, and Value in Self-Attention

In self-attention, each input token is transformed into three vectors:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- Q = Query vector
- K = Key vector
- V = Value vector

These vectors are learned projections of the same input X , using different weight matrices.

In self-attention, the query, key, and value vectors all come from the same input sequence X . In cross-attention, the query comes from one sequence, while the keys and values come from another sequence:

$$Q = X_{\text{decoder}} W^Q, \quad K = X_{\text{encoder}} W^K, \quad V = X_{\text{encoder}} W^V$$

Query, Key, and Value in Attention

- In self-attention, each token is represented in **three different roles**:
 - **Query (Q)**: asking for information *“Who can help me?”*
 - **Key (K)**: offering information *“I have some info, check if you need me.”*
 - **Value (V)**: the actual information *“Here’s the content I can share.”*
- Computation:
 - Compare **Query** of one token with all **Keys** → attention scores.
 - Convert scores to weights (softmax).
 - Use weights to mix the **Values** → new representation of the token.
- Effect: each word updates its meaning by gathering info from all the others.

Each vector receives three representations (“roles”)

$$[W_Q] \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Query: vector **from** which the attention is looking

“Hey there, do you have this information?”

$$[W_K] \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Key: vector **at** which the query looks to compute weights

“Hi, I have this information – give me a large weight!”

$$[W_V] \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Value: their weighted sum is attention output

“Here’s the information I have!”

figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Intuition Behind Query, Key, and Value

- **Query** (Q) – represents what a token is *looking for*. Think: “Who is related to me? Who should I pay attention to?”
- **Key** (K) – represents the properties of a token that can be matched against a query. Think: “I carry certain features that others may look for.”
- **Value** (V) – contains the actual information to be passed along once a match is found. Think: “If you attend to me, this is the content I contribute.”

Attention works by matching Queries against Keys, and then using the corresponding Values.

Example of Queries, Keys, and Values

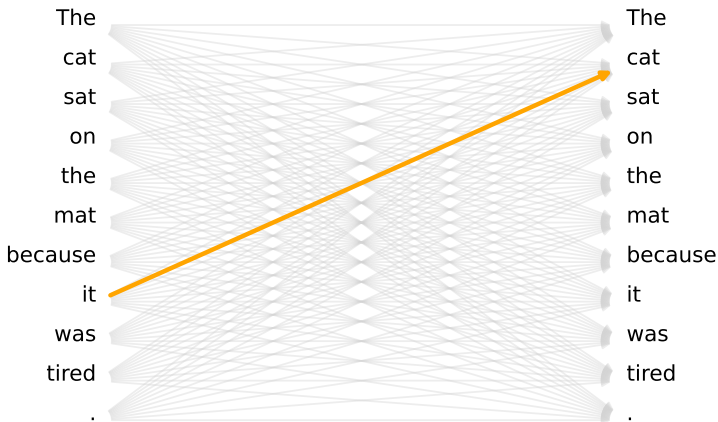
Consider the sentence:

“The cat sat on the mat because it was tired.”

- The word **“it”** forms a **Query** asking: “Who am I referring to?”
- The word **“cat”** provides a matching **Key**, signaling: “I am a possible referent.”
- The **Value** from **“cat”** is passed to **“it”**, allowing the model to connect the pronoun to the correct noun.

Visual Example

Self-Attention Example: 'The cat sat on the mat because it was tired.'



Self-Attention Formula

The attention mechanism works by matching **Queries** with **Keys** and retrieving the corresponding **Values**:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- QK^T computes similarity between queries and keys.
- softmax normalizes these scores into attention weights.
- The weights are applied to V to produce the output representation.

Scaled Dot-Product Attention

- Attention takes three inputs: **Query (Q)**, **Key (K)**, **Value (V)**.
- Step 1: Compute similarity between query q and all keys k :

$$qk^{\top}$$

- Step 2: Scale by $\sqrt{d_k}$ (key dimension) to avoid very large values.
- Step 3: Apply softmax $\rightarrow$ attention weights (probability distribution).
- Step 4: Use weights to mix the values $v \rightarrow$ new representation.

$$\text{Attention}(q, k, v) = \text{softmax}\left(\frac{qk^{\top}}{\sqrt{d_k}}\right) v$$

Intuition: Compare Q with K to decide *where to look*, then use V to decide *what information to take*.

Why Scale by $\sqrt{d_k}$?

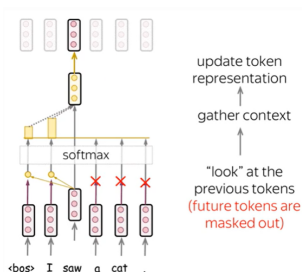
- Attention uses a softmax over dot products:

$$\text{softmax}\left(\frac{qk^\top}{\sqrt{d_k}}\right)$$

- The key idea is that d_k tells us how many numbers are being multiplied and added together in the dot product.
- Problem: without scaling, $qk^\top$ grows with d_k : Assuming the components of q and k are approximately independent, zero-mean, and unit-variance, the variance of $qk^\top$ grows with d_k , so its typical magnitude grows as $\sqrt{d_k}$.
 - For large d_k , dot products become very large.
 - Softmax of large values $\rightarrow$ nearly one-hot outputs.
 - This makes gradients vanish $\rightarrow$ training becomes unstable.
- Why the $\sqrt{d_k}$ specifically?
 - The dot product is a sum of d_k terms, so its variance grows with d_k .
 - Standard deviation grows as $\sqrt{d_k}$.
 - Dividing by $\sqrt{d_k}$ keeps the values in a moderate range.
- **Effect:** softmax stays smooth and informative, avoiding overconfidence and stabilizing training.

Masked Self-Attention in the Decoder

- Encoder self-attention: each token can look at **all tokens** in the input.
- Decoder self-attention: we generate tokens **one at a time**.
 - During generation, future tokens are unknown.
 - To prevent “cheating,” the decoder is restricted to look only at **previous tokens**.
- Solution: **Masked self-attention**
 - Masks out all future positions.
 - Ensures autoregressive generation.



Multi-Head Attention: Focus on Different Things

Multi-Head Attention extends self-attention by using multiple sets of Queries, Keys, and Values.

- One attention head learns one “relation” between tokens.
- But words often play many roles, for example grammar, semantics, agreement.
- Instead of computing attention once, the model projects the input into multiple “heads”.
- Each head learns different relationships, for example syntax, semantics, long-range dependencies.
- **Multi-head attention:**
 - Run several attention mechanisms, or heads, in parallel.
 - Each head focuses on different aspects of context.
 - Outputs are concatenated and transformed into one representation.
- The outputs of all heads are concatenated and combined to form the final representation.
- Effect: richer understanding of how words relate to each other.

$$\text{Multi-Head Attention}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) = \text{softmax}\left(\frac{(QW_i^Q)(KW_i^K)^T}{\sqrt{d_k}}\right) (VW_i^V)$$

Multi-Head Attention

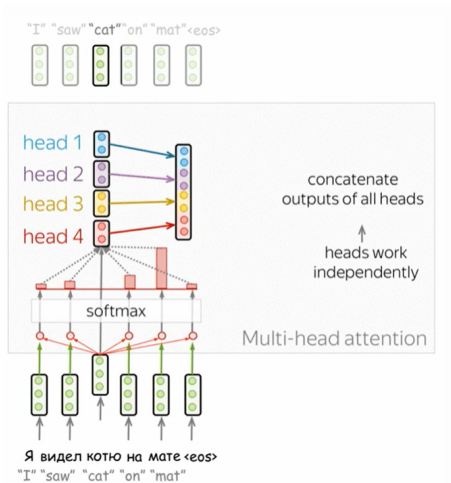
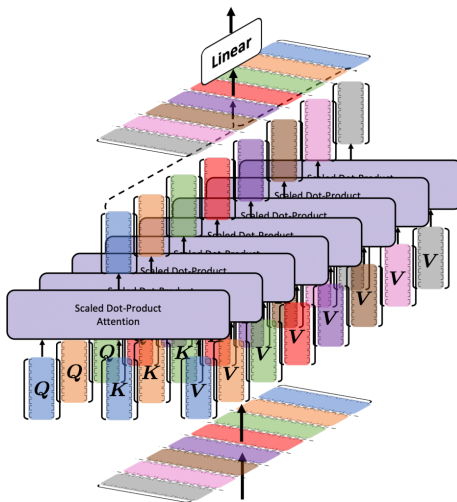


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Multi-Headed Self-Attention



Source: Multi-head Attention Mechanism

Why having multiple heads?

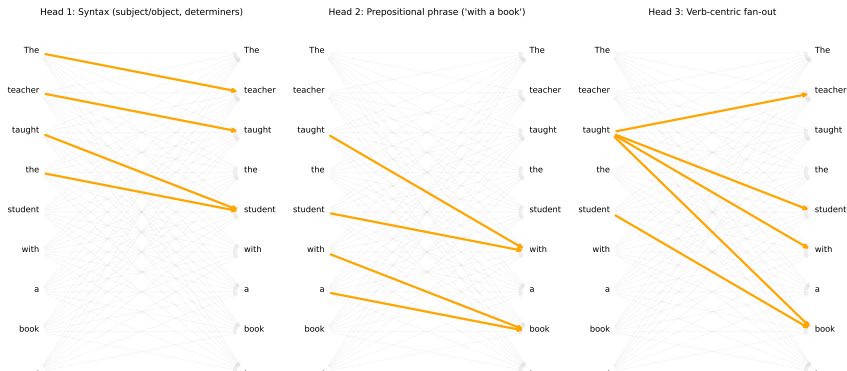
Multi-head attention allows the model to capture different types of information in parallel.

Different Heads: Different Patterns

Multi-Head Attention: Example

Each head can specialize in a different aspect of the sentence.

Multi-Head Attention: Different Heads, Different Patterns



More Ingredients in Transformer Layers

Multi-head attention is the core part of a Transformer, but it is not the only component.

A Transformer layer also uses:

- **FFN**
- **Residual connections**
- **Layer normalization**
- **Positional encoding**

Feed-Forward Neural Network

In Transformers, the FFNN expands each token representation into a larger feature space, applies a nonlinear transformation to detect useful patterns, then compresses it back to the original size. It is like **massaging the data!** After attention, each token's representation is passed through a **Feed-Forward Network (FFNN)**:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Same FFN applied independently to each position.
- Think of it as: “take the gathered info, and process it further.”

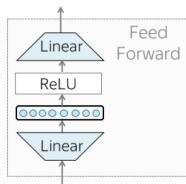


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Residual connections

Residual connections:

- Add input of a block to its output.
- Helps gradient flow → makes stacking many layers possible.

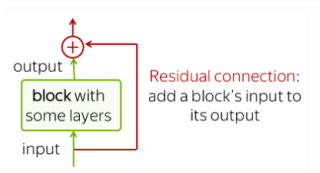


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Residual Connections in Transformers

Residual connections are applied around each main sub-layer.

- Around **multi-head attention**:

$$x_{\text{attn}} = x + \text{MultiHeadAttention}(x)$$

- Around the **feed-forward network**:

$$x_{\text{out}} = x_{\text{attn}} + \text{FFN}(x_{\text{attn}})$$

- They help the model preserve the original information while learning useful changes.
- They also help gradients flow through many stacked layers.

Main idea: each sub-layer learns an update, not a completely new representation.

Layer Normalization

Layer Normalization:

- Normalizes token representations within each layer.
- Stabilizes training and improves convergence.

Where it is applied:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- The residual connection adds the input back.
- Layer normalization rescales the result.
- This keeps values controlled as they move through many Transformer layers.

Main idea: keep each token representation stable before passing it to the next layer.

Positional Encoding

Positional Encoding:

- Adds word-order information to token embeddings.
- Needed because self-attention does not naturally know token positions.
- Helps the model distinguish sentences with the same words in different orders.

Transformer input = Token Embedding + Positional Encoding

- Token embedding tells the model **what** the word is.
- Positional encoding tells the model **where** the word is.

Main idea: attention needs position information to understand word order.

Understanding Positional Encoding Formulas

Goal: give each token a unique position pattern.

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos = position of the token in the context window (current input sequence).
- i = dimension index inside the embedding vector.
- d_{model} = total embedding size.
- Even dimensions use $\sin$.
- Odd dimensions use $\cos$.
- Different dimensions use different wavelengths, so the model can represent both short-range and long-range positions.

Main idea: each word gets its meaning embedding plus a position pattern.

How to Calculate Positional Encoding

Suppose the current input sequence is:

[The, cat, sat, on]

Positions: 0, 1, 2, 3

For the word “cat”, we have:

$$pos = 1$$

If the embedding size is:

$$d_{\text{model}} = 4$$

then we calculate four position values:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

For “cat”, where $pos = 1$:

$$PE_1 = \left[\sin(1), \cos(1), \sin\left(\frac{1}{100}\right), \cos\left(\frac{1}{100}\right) \right]$$

Final input vector = token embedding of “cat” + PE_1

Tokens at the same position across different batches receive the same positional encoding, but their final input vectors are different because their token embeddings are different.

Note: Each token is assigned a unique position within the sequence, starting from 0 and increasing up to the model’s maximum context length. Once this context-length limit is reached, the model cannot process additional positions within the same input sequence.

Why Transformers Changed NLP

Advantages over previous models:

- **Parallel processing:** All tokens processed simultaneously — faster training.
- **Long-range dependencies:** Attention connects distant words directly.
- **Scalability:** Scales efficiently to large data and deep architectures.
- **Transfer learning:** Enables pretraining and fine-tuning (e.g., BERT, GPT).

Impact:

- Became the foundation of modern NLP systems.
- Used in models like GPT, BERT, T5, and many others.

Transformers replaced recurrence with attention, ushering in the era of large-scale, context-aware language models.

Current Section

- 1 Attention
- 2 Transformer
- 3 Encoder-Decoder Overview**

Transformer Architecture

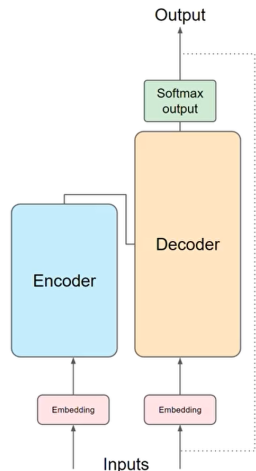
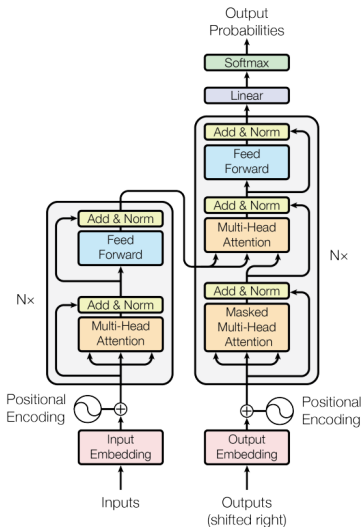


figure source: <https://www.coursera.org/learn/generative-ai-with-llms/home/module/1>

Encoder-Decoder Overview

The original Transformer has two main components: an **encoder** and a **decoder**.

Encoder: processes the source sequence, such as an English sentence. It uses **self-attention**, so each input token can attend to the other input tokens.

Decoder: generates the target sequence, such as a French sentence, one token at a time. It uses two attention mechanisms:

- **Masked self-attention:** attends to previously generated target tokens.
- **Cross-attention:** attends to the encoder's hidden states from the source sentence.

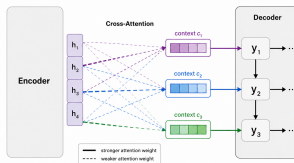
The encoder produces one hidden state per input token. The decoder uses these hidden states to generate the output sequence.

Context Vector

In encoder-decoder models, the decoder needs information from the source sentence. This information is often called the **context**.

- **Early RNN seq2seq:** the last encoder hidden state is used as one context vector representing the whole input sentence.
- **With attention:** a new context vector is created **at each decoding step**. In Transformers, the encoder produces hidden states for all input tokens, and the decoder uses **cross-attention** to build a context vector for each generated token.

Main idea: the context is not one fixed summary. It changes depending on which output token the decoder is generating.



A new context vector is built at each decoding step.

Encoder and Decoder

Neural sequence models do not always have both an encoder and a decoder. The architecture depends on the task.

- **Encoder-only models:** used when the model needs to understand an input.
 - Example: text classification, sentiment analysis, sentence embeddings.
 - Common architecture: BERT-style models.
- **Encoder-decoder models:** used when the model transforms one sequence into another.
 - Example: translation, summarization, question answering.
 - The encoder processes the input. The decoder generates the output.
- **Decoder-only models:** used when the model generates text step by step.
 - Example: language generation, chatbots, next-word prediction.
 - Common architecture: GPT-style models.

Main idea: an encoder or decoder may exist or may not exist, depending on the use case and model architecture.

Encoder-Only Models (Autoencoding)

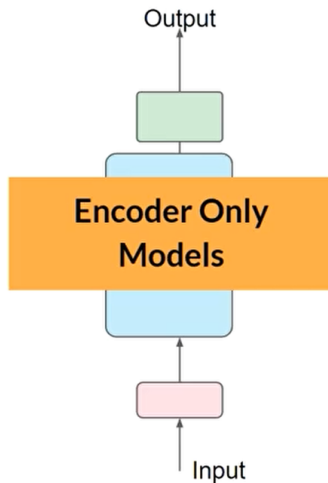


figure source: <https://www.coursera.org/learn/generative-ai-with-llms/home/module/1>

Encoder-Only Models (Autoencoding)

Examples: BERT, RoBERTa, DistilBERT

These models are all part of the **BERT family**.

- Use **masked language modeling (MLM)** objective, where some tokens are hidden (masked) in the input and the model predicts them using surrounding context.
- Input text is partially masked, model reconstructs the missing words.
- Learns **bidirectional context**.
- Great for: sentiment analysis, named entity recognition, text classification.

Encoder-Only Objective: Masked Language Modeling (MLM)

Used in: Encoder-only models (BERT, RoBERTa, DistilBERT)

- Randomly masks some tokens in the input.
- Model predicts the missing tokens using both left and right context.
- Encourages learning of **bidirectional context**, meaning that in self-attention, each token can attend to both previous and future tokens.
- MLM (BERT): fill in random single words → best for understanding tasks.

Example:

Input: The teacher [MASK] the student.

Target: The teacher **teaches** the student.

Encoder–Decoder Models (Seq2Seq)

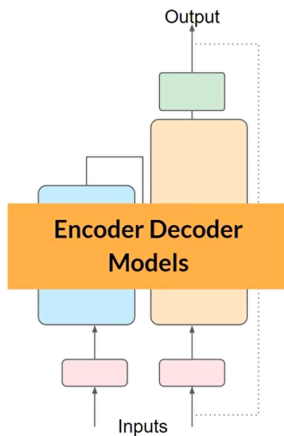


figure source: <https://www.coursera.org/learn/generative-ai-with-llms/home/module/1>

Encoder–Decoder Models (Seq2Seq)

Examples: T5, BART

- Use **sequence-to-sequence** objectives (e.g., span corruption), where parts of the input text are replaced with sentinel tokens and the model learns to reconstruct the missing spans.
- Encoder reads input and produces hidden states.
- Decoder generates output conditioned on encoder states (**cross-attention**).
- Great for: translation, summarization, question answering.

Encoder–Decoder Objective: Span Corruption

Used in: Encoder–decoder models (T5, BART)

- Replaces contiguous spans of text with sentinel tokens.
- Encoder sees text with gaps; decoder reconstructs the missing spans.
- Suitable for tasks like summarization, translation, question answering.
- Span Corruption (T5): fill in phrases/spans → best for generating coherent text (sequence-to-sequence tasks).

Example:

Input to encoder: The teacher <X> student.

Target for decoder: <X> teaches the

This means the encoder receives a sentence with a missing phrase marked by a sentinel token, and the decoder learns to generate the missing phrase that should replace that token.

Decoder-Only Models (Autoregressive)

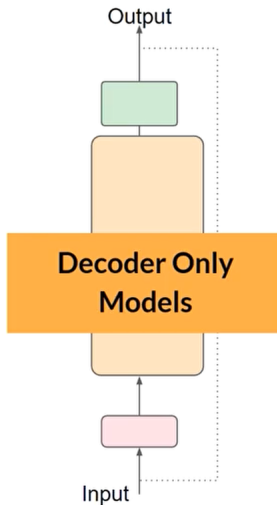


figure source: <https://www.coursera.org/learn/generative-ai-with-llms/home/module/1>

Decoder-Only Models (Autoregressive)

Examples: GPT, LLaMA, BLOOM

- Use **causal language modeling (CLM)** objective, where the model predicts the next token in a sequence given only the past tokens (no future context).
- Predicts the next token given all previous ones.
- Learns **unidirectional context**.
- Great for: text generation, story writing, code generation, dialogue.

Decoder-Only Objective: Causal Language Modeling (CLM)

Used in: Decoder-only models (GPT, LLaMA, BLOOM)

- Predicts the next token given all previous tokens.
- Model only has access to the **past**, not future tokens.
- Encourages learning of **unidirectional context**.
- CLM (GPT): Predicts the next token given past tokens → best for text generation.

Example:

Input: The teacher

Target: The teacher **teaches**

Observation: Decoder-Only Models

Although each architecture type was originally designed for specific tasks:

- **Encoder-only** → understanding tasks (e.g., classification, NER, sentiment analysis)
- **Encoder-decoder** → sequence-to-sequence tasks (e.g., translation, summarization, QA)
- **Decoder-only** → generative tasks (e.g., text generation, dialogue, code)

But in practice:

- Large decoder-only models (e.g., GPT, LLaMA) perform strongly across **all task types**.
- With prompting or fine-tuning, they can handle classification, translation, summarization, reasoning, etc.

Conclusion: Decoder-only models appear to be highly versatile — suggesting that in many applications the encoder part may not be strictly necessary.

Note: Decoder-Only Models, Why Predict One Token at a Time?

Have you ever wondered why we predict one word at a time instead of the whole sentence at once?

We predict one word at a time because predicting an entire sentence at once would require estimating an impossible number of combinations, while step-by-step prediction keeps the model tractable and context-aware.

Why Not Model Whole Sentences Directly?

- A sentence of length n from a vocabulary of size $|V|$ has $|V|^n$ possible combinations.
 - Example: $|V| = 50,000$, $n = 10$
 - $\Rightarrow 50,000^{10} \approx 10^{44}$ possible sentences!
- Impossible to store or estimate probabilities for all sequences.

Solution: Factorization

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

Instead of modeling full sentences, we model word-by-word probabilities.

Congratulations!

Now you know how attention overcomes the fixed-vector bottleneck!

